Programming Fundamentals

Week 7 — Relative File Paths and `main()`

Programming Fundamentals Team

SETU

2026-10-28

Two Small Ideas That Make Programs Easier to Run

Today we will learn:

- where Python looks for files
- how to use **relative file paths**
- how to organise a program around a `main()` function
- what `if __name__ == "__main__":` means

These ideas will help us organise the PC Troubleshooter and its JSON knowledge file.

A Familiar Problem

You have a file called `knowledge.json`.

```
with open("knowledge.json", "r") as file:
    data = file.read()
```

But Python says: `FileNotFoundError`.

A Familiar Problem

You have a file called `knowledge.json`.

```
with open("knowledge.json", "r") as file:
    data = file.read()
```

But Python says: `FileNotFoundError`.

The file exists, so why might Python not find it?

A Project Has Folders and Files

```
pc_troubleshooter/
|-- troubleshooter.py
|-- knowledge.json
|-- data/
    |-- history.json
```

Files have **locations**, not just names.

The path tells Python which file to use.

Absolute and Relative Paths

Absolute path: starts from a fixed location on the computer.

```
C:\\Users\\Student\\Projects\\pc_troubleshooter\\knowledge.json
```

Relative path: starts from the program's **current working directory**.

```
knowledge.json
data/history.json
```

Relative paths make projects easier to move between computers — provided the working directory is set correctly.

Where Does Python Start Looking?

A relative path is normally resolved from the **current working directory** (CWD).

```
import os

print(os.getcwd())
```

The CWD is not necessarily the folder containing the Python script.

Where Does Python Start Looking?

A relative path is normally resolved from the **current working directory** (CWD).

```
import os

print(os.getcwd())
```

The CWD is not necessarily the folder containing the Python script.

What happens if you run the script from a different folder?

Try It: Read a File in the Same Folder

If the working directory is `pc_troubleshooter/`:

```
with open("knowledge.json", "r") as file:
    contents = file.read()

print(contents)
```

Live task: Put a small text or JSON file in your project folder and read it.

Try It: Read a File in a Subfolder

To access `data/history.json`:

```
with open("data/history.json", "r") as file:
    contents = file.read()
```

The `/` separates folders in a path.

Use forward slashes in these examples; they also work in Python paths on Windows.

What Does `..` Mean?

Relative paths can include:

- `.` — the current directory
- `..` — the parent directory

```
../knowledge.json
```

This means: **go up one folder, then find `knowledge.json`.**

What Does `..` Mean?

Relative paths can include:

- `.` — the current directory
- `..` — the parent directory

```
../knowledge.json
```

This means: **go up one folder, then find `knowledge.json`.**

Question: Would this work from every working directory?

A More Reliable Approach: `pathlib`

Sometimes we want a path relative to the **script**, not the CWD.

```
from pathlib import Path

folder = Path(__file__).resolve().parent
knowledge_file = folder / "knowledge.json"

print(knowledge_file)
```

`Path(__file__)` refers to the Python file being run.

Useful for Assignment 2: keep your JSON file beside your Python file.

Read JSON Using a Script-Relative Path

```
import json
from pathlib import Path

folder = Path(__file__).resolve().parent
filename = folder / "knowledge.json"

with open(filename, "r", encoding="utf-8") as file:
    knowledge = json.load(file)
```

Note: `json.load()` reads JSON and converts it to Python data.

Quick Check: Which Path Would You Use?

```
project/
|-- app.py
|-- rules.json
|-- data/
    |-- history.json
```

Assume the CWD is `project/`.

1. How do we open `rules.json`?
2. How do we open `history.json`?
3. What changes if the CWD is the folder above `project/`?

Quick Check: Which Path Would You Use?

```
project/
|-- app.py
|-- rules.json
|-- data/
    |-- history.json
```

Assume the CWD is `project/`.

1. How do we open `rules.json`?
2. How do we open `history.json`?
3. What changes if the CWD is the folder above `project/`?

Answers: `rules.json`, `data/history.json`; from the parent use `project/rules.json` and `project/data/history.json`.

Next: Where Should Our Program Start?

So far, we have written functions such as:

```
def display_menu():
    print("1. Start troubleshooting")
    print("2. Exit")
```

But where do we put the code that **runs the whole program**?

> We can create a `main()` function to organise the starting point.

Introducing `main()`

```
def display_menu():
    print("Welcome to PC Troubleshooter")

def main():
    display_menu()
    print("Ready to begin!")

main()
```

`main()` is an ordinary function — we choose that name by convention.

Why Use a `main()` Function?

It helps us:

- see where the program starts
- keep the main sequence of steps together
- avoid a long block of loose code
- reuse and test other functions separately

Think of `main()` as the organiser that calls the other functions.

A Typical Menu-Driven `main()`

```
def main():
    choice = ""
    while choice != "2":
        print("1. Troubleshoot a problem")
        print("2. Exit")
        choice = input("Choose an option: ")

        if choice == "1":
            print("Starting troubleshooting...")

main()
```

Question: Which parts would you move into separate functions?

A Special Line You Will Often See

```
if __name__ == "__main__":
    main()
```

Python automatically sets the variable `__name__`.

When we run a file directly, its `__name__` is `"__main__"`.

This line means: **run `main()` only if this file was started directly.**

Why Not Just Write `main()`?

Imagine another Python file imports your functions.

If you write `main()` at the bottom, the program starts **during the import**.

If you write:

```
if __name__ == "__main__":
    main()
```

the functions can be imported without automatically running the menu.

You don't need to memorise every detail yet — recognise this as the standard way to start a Python program.

Our PC Troubleshooter: Putting It Together

```
import json
from pathlib import Path

def load_knowledge():
    path = Path(__file__).resolve().parent / "knowledge.json"
    with open(path, "r", encoding="utf-8") as file:
        return json.load(file)

def main():
    knowledge = load_knowledge()
    print("Knowledge loaded successfully!")

if __name__ == "__main__":
    main()
```

Your Turn: Improve the Program

Working with the PC Troubleshooter:

1. Put `knowledge.json` beside your Python file.
2. Create a `load_knowledge()` function.
3. Add a `main()` function that calls it.
4. Add the standard `if __name__ == "__main__":` line.
5. Run the program and check that the file loads.

Challenge: Move the JSON file into a `data/` folder and update the path.

Before We Finish

You should now be able to explain:

- the difference between an absolute and relative path
- what the current working directory is
- why `Path(__file__).resolve().parent` is useful
- what a `main()` function does
- why we use `if __name__ == "__main__":`

Next week: debugging and exception handling — including what to do when a file is missing or contains invalid data.